

CRRSS Phase 2 Stage 5: Enhanced Build System Integration Guide

Overview

Phase 2 Stage 5 completes the CRRSS framework integration into the BDI repository by implementing a comprehensive build system with:

- **Enhanced Pre-commit Hooks:** Automated code quality checks with multiple profiles
- **Updated Makefile Targets:** Comprehensive build, test, and CI/CD integration
- **CI/CD Pipeline:** GitHub Actions workflows for automated testing and deployment
- **System Testing:** Comprehensive test suite covering all CLI commands and edge cases
- **Documentation:** Complete guide to the build system and CI/CD integration

Table of Contents

1. [Pre-commit Hooks](#)
 2. [Build System](#)
 3. [Testing Framework](#)
 4. [CI/CD Pipeline](#)
 5. [Usage Examples](#)
 6. [Troubleshooting](#)
-

Pre-commit Hooks

Overview

The CRRSS pre-commit hooks provide automated code quality checks before each commit, ensuring code meets project standards.

Features

- **CRRSS Static Analysis:** Analyze staged C/H files for potential issues
- **Code Formatting:** Check code formatting with clang-format
- **Syntax Checking:** Validate C syntax with gcc
- **Test Execution:** Optionally run test suite before commit
- **Build Profile Support:** Support for debug and release profiles
- **Configurable Strictness:** Warning mode (default) or strict mode

Installation

```
# From repository root
./git-hooks/install-hooks.sh install

# Check status
./git-hooks/install-hooks.sh status
```

Configuration

Configure hook behavior with environment variables:

CRRSS_ENABLED

Enable/disable the hook:

```
export CRRSS_ENABLED=1 # Enabled (default)
export CRRSS_ENABLED=0 # Disabled
```

CRRSS_STRICT

Control blocking behavior:

```
export CRRSS_STRICT=0 # Warning mode - show issues but allow commit (default)
export CRRSS_STRICT=1 # Strict mode - block commit on critical issues
```

CRRSS_BUILD_PROFILE

Set build profile:

```
export CRRSS_BUILD_PROFILE=release # Release build (default)
export CRRSS_BUILD_PROFILE=debug # Debug build
```

CRRSS_RUN_TESTS

Enable test execution:

```
export CRRSS_RUN_TESTS=0 # Don't run tests (default)
export CRRSS_RUN_TESTS=1 # Run tests before commit
```

CRRSS_CHECK_FORMAT

Enable code formatting checks:

```
export CRRSS_CHECK_FORMAT=1 # Check formatting (default)
export CRRSS_CHECK_FORMAT=0 # Skip formatting checks
```

CRRSS_CHECK_SYNTAX

Enable syntax checking:

```
export CRRSS_CHECK_SYNTAX=1 # Check syntax (default)
export CRRSS_CHECK_SYNTAX=0 # Skip syntax checks
```

Usage Examples

Development Workflow

```
# Regular development (warnings only)
git commit -m "Add feature"

# Enable strict mode for important commits
CRRSS_STRICT=1 git commit -m "Critical fix"

# Run tests before commit
CRRSS_RUN_TESTS=1 git commit -m "Add feature with tests"

# Quick commit without checks
CRRSS_ENABLED=0 git commit -m "WIP: work in progress"
```

Team Configuration

Add to `.bashrc` or `.zshrc`:

```
# Strict mode for all commits
export CRRSS_STRICT=1

# Run tests before commit
export CRRSS_RUN_TESTS=1

# Use debug profile
export CRRSS_BUILD_PROFILE=debug
```

Build System

Overview

The CRRSS build system provides comprehensive Makefile targets for building, testing, and integrating CRRSS into the BDI project.

Build Modes

Release Mode (Default)

```
make all BUILD_MODE=release
```

- Optimized for performance (`-O3`)
- Link-time optimization (`-flto`)
- Architecture-specific optimizations (`-march=native`)

Debug Mode

```
make all BUILD_MODE=debug
```

- No optimization (`-O0`)
- Full debug symbols (`-g3`)
- AddressSanitizer and UndefinedBehaviorSanitizer enabled

- Useful for development and debugging

Makefile Targets

Build Targets

```
# Build everything (library, tool, tests)
make all

# Build CRRSS library only
make lib

# Build CRRSS CLI tool only
make tool

# Build test suite only
make tests
```

Test Targets

```
# Run all unit tests
make test

# Run system tests (comprehensive CLI testing)
make system-test

# Run integration tests
make integration-test

# Run specific component tests
make test-bpme
make test-sciv
make test-memory
make test-msm
make test-stp
make test-tdt
make test-cli
```

CI/CD Targets

```
# CI build (clean + build)
make ci-build

# CI test (build + tests + system tests)
make ci-test

# Full CI pipeline
make ci-all
```

Code Quality Targets

```
# Format code with clang-format
make format

# Check code formatting
make check-format
```

Analysis Targets

```
# Analyze BDI codebase with CRRSS
make check-codebase

# Analyze specific file
make analyze-file FILE=path/to/file.c
```

Installation Targets

```
# Install CRRSS to system (default: /usr/local)
make install

# Install to custom location
make install INSTALL_PREFIX=/opt/crrss

# Uninstall CRRSS
make uninstall
```

Other Targets

```
# Clean build artifacts
make clean

# Validate build
make validate

# Show build information
make info

# Show help
make help
```

Root Makefile Integration

CRRSS is fully integrated into the root BDI Makefile:

```
# From repository root
make crrss           # Build CRRSS
make crrss-test      # Run CRRSS tests
make crrss-system-test # Run system tests
make crrss-integration-test # Run integration tests
make crrss-ci-all    # Run full CI pipeline
make crrss-check      # Analyze BDI codebase
make crrss-info       # Show CRRSS info
make crrss-help       # Show CRRSS help
```

Testing Framework

Overview

The CRRSS testing framework includes three levels of testing:

1. **Unit Tests:** Test individual components (BPME, SCIV, MSM, STP, TDT, CLI)

2. **System Tests:** Comprehensive CLI testing with edge cases
3. **Integration Tests:** Test CRRSS integration with BDI codebase

Unit Tests

Located in `tests/` directory:

- `test_bpme.c` - Bug Prior Mapping Engine tests
- `test_sciv.c` - Self-Check Internal Validator tests
- `test_memory.c` - Memory Integration Layer tests
- `test_msm.c` - Memory-Safety Maniac tests
- `test_stp.c` - Strategic Timmy Profile tests
- `test_tdt.c` - Test-Driven Timmy tests
- `test_cli.c` - CLI functionality tests

Run unit tests:

```
make test
```

System Tests

Comprehensive test suite covering:

- All CLI commands (query, msm, stp, validate)
- Buffer overflow security fixes (PR#182)
- Edge cases and error handling
- Report format generation (text, JSON, HTML)
- Component integration
- Performance testing

Run system tests:

```
./tests/system_test.sh  
# or  
make system-test
```

Integration Tests

Test CRRSS integration with BDI:

- CLI tool functionality
- BDI codebase analysis
- Build system integration

Run integration tests:

```
make integration-test
```

Test Output

Test results include:

- **Total Tests:** Number of tests executed
- **Passed:** Number of successful tests
- **Failed:** Number of failed tests
- **Detailed Results:** Individual test status

Example output:

```
=====
CRRSS System Test Suite
=====

[TEST] Display help message
[PASS] Display help message

[TEST] Query by priority P0
[PASS] Query by priority P0

...

=====
Test Summary
=====
Total Tests: 45
Passed: 45
Failed: 0

=====
All tests passed! ✓
=====
```

CI/CD Pipeline

Overview

The CRRSS CI/CD pipeline uses GitHub Actions to automate building, testing, and deployment.

Workflows

1. CRRSS CI/CD Pipeline (`.github/workflows/crrss-ci.yml`)

Triggers:

- Push to main, develop, feature/, bugfix/ branches
- Pull requests to main, develop
- Manual workflow dispatch

Jobs:

- **build-and-test**: Build CRRSS in debug and release modes, run tests
- **code-quality**: Check code formatting, run static analysis
- **security-scan**: Scan for security vulnerabilities
- **integration-test**: Test CRRSS integration with BDI
- **documentation**: Check documentation completeness
- **notify-status**: Report pipeline status

Features:

- Matrix build (debug/release)
- Memory leak detection (debug mode)
- Code formatting checks (clang-format)
- Static analysis (clang-tidy, cppcheck)
- Security scanning
- Artifact uploads

2. CRRSS Release (`.github/workflows/crrss-release.yml`)

Triggers:

- Release creation/publication
- Manual workflow dispatch

Jobs:

- **build-release**: Build release artifacts for multiple platforms
- Package CRRSS tool and documentation
- Upload release assets

3. CRRSS Nightly Build (`.github/workflows/crrss-nightly.yml`)

Triggers:

- Daily at 2 AM UTC
- Manual workflow dispatch

Jobs:

- **nightly-build**: Build and test CRRSS
- Run comprehensive tests
- Memory leak detection
- Static analysis
- BDI codebase analysis
- Generate nightly reports

CI/CD Usage

Running Locally

Simulate CI/CD pipeline locally:

```
# Build
make crrss-ci-build

# Test
make crrss-ci-test

# Full pipeline
make crrss-ci-all
```

Viewing Results

- **GitHub Actions**: View workflow runs in repository Actions tab
- **Artifacts**: Download build artifacts and test reports
- **Logs**: View detailed logs for each workflow step

CI/CD Best Practices

1. **Commit Often**: Make small, focused commits
 2. **Test Locally**: Run `make crrss-ci-all` before pushing
 3. **Fix Failures**: Address CI failures promptly
 4. **Review Artifacts**: Check test reports and analysis results
 5. **Use Pre-commit Hooks**: Enable strict mode for critical branches
-

Usage Examples

Example 1: Development Workflow

```
# 1. Install pre-commit hooks
./git-hooks/install-hooks.sh install

# 2. Configure development environment
export CRRSS_BUILD_PROFILE=debug
export CRRSS_STRICT=0
export CRRSS_RUN_TESTS=0

# 3. Make code changes
vim tools/crrss/cli/crrss_cli.c

# 4. Build and test locally
make crrss-ci-all

# 5. Commit changes
git add .
git commit -m "Add new feature"
# Pre-commit hooks run automatically

# 6. Push changes
git push origin feature/my-feature
```

Example 2: Production Release

```
# 1. Switch to release mode
export CRRSS_BUILD_PROFILE=release
export CRRSS_STRICT=1
export CRRSS_RUN_TESTS=1

# 2. Run full test suite
make crrss-ci-all

# 3. Build release artifacts
make clean
make all BUILD_MODE=release

# 4. Install to system
sudo make install

# 5. Verify installation
crrss --version
```

Example 3: CI/CD Integration

```
# .github/workflows/my-workflow.yml
name: My Workflow

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build CRRSS
        run: make crrss-ci-build

      - name: Run CRRSS tests
        run: make crrss-ci-test

      - name: Analyze codebase
        run: make crrss-check
```

Example 4: Code Quality Checks

```
# Check code formatting
make check-format

# Format code automatically
make format

# Run static analysis
make crrss-check

# Analyze specific file
make analyze-file FILE=moduler_kernel/memory.c
```

Troubleshooting

Common Issues

1. Pre-commit Hook Not Running

Problem: Hook doesn't execute on commit

Solution:

```
# Check if hook is installed
ls -la .git/hooks/pre-commit

# Make hook executable
chmod +x .git/hooks/pre-commit

# Reinstall hooks
./git-hooks/install-hooks.sh install
```

2. CRRSS Tool Not Found

Problem: CRRSS tool not found error

Solution:

```
# Build CRRSS tool
make crrss-tool

# Or build everything
make crrss
```

3. System Tests Failing

Problem: System tests fail to run

Solution:

```
# Make system test script executable
chmod +x tools/crrss/tests/system_test.sh

# Rebuild CRRSS
cd tools/crrss
make clean
make all

# Run tests
./tests/system_test.sh
```

4. CI/CD Pipeline Failures

Problem: GitHub Actions workflow fails

Solution:

```
# Run CI pipeline locally
make crrss-ci-all

# Check specific failures
make crrss-ci-build # Test build
make crrss-ci-test  # Test suite

# View logs in GitHub Actions tab
```

5. Memory Leak Detected

Problem: Valgrind reports memory leaks

Solution:

```
# Build in debug mode
make clean
make all BUILD_MODE=debug

# Run with valgrind
valgrind --leak-check=full ./build/bin/crrss --help

# Fix leaks in code
# Rebuild and test
```

6. Code Formatting Issues

Problem: Formatting checks fail

Solution:

```
# Check which files need formatting
make check-format

# Format all files automatically
make format

# Commit formatted files
git add .
git commit -m "Fix code formatting"
```

Getting Help

- **Documentation:** See `tools/crrss/docs/` for detailed guides
- **Makefile Help:** Run `make crrss-help` or `make help`
- **Pre-commit Hook Help:** Run `./.git-hooks/install-hooks.sh --help`
- **System Test Logs:** Check `/tmp/crrss_system_test_*/` for test artifacts

Performance Tips

1. **Use Release Mode:** Build with `BUILD_MODE=release` for faster execution
2. **Parallel Builds:** Use `make -j$(nproc)` for faster compilation
3. **Skip Tests:** Use `CRRSS_RUN_TESTS=0` for faster commits during development
4. **Cache Builds:** CI/CD uses artifact caching for faster pipelines

Summary

Phase 2 Stage 5 provides:

- ✓ **Enhanced Pre-commit Hooks:** Automated code quality with multiple profiles
- ✓ **Comprehensive Build System:** Make targets for all build, test, and CI/CD operations
- ✓ **System Testing:** Complete test coverage for CLI commands and edge cases
- ✓ **CI/CD Pipeline:** GitHub Actions workflows for automated testing and deployment
- ✓ **Documentation:** Complete guide to build system and CI/CD integration

The CRRSS framework is now fully integrated into the BDI project with a robust build system and comprehensive testing infrastructure.

Version: 1.0.0

Phase: 2 Stage 5

Date: October 2025

Author: BDI/CRRSS Team